

Nama : Syahril Arfian Almazril
Nim : 103032300013
Kelas : IT-47-04
Kode Asprak : STO

A. Teori

1. Apa yang dimaksud dengan web server? Jelaskan bagaimana sebuah web server berkomunikasi dengan klien menggunakan protokol HTTP! Apa peran socket dalam proses ini? dan mengapa TCP menjadi pilihan utama untuk komunikasi antara web server dan klien?

Web server adalah sistem yang bertugas untuk menyimpan dan mengirimkan konten web kepada klien, seperti browser, menggunakan protokol HTTP. Jadi Ketika klien mengirimkan permintaan untuk mengakses halaman atau file tertentu, server akan memprosesnya dan mengirimkan respons yang sesuai, seperti halaman HTML atau gambar. Protokol HTTP memastikan komunikasi ini berjalan dengan baik, sementara socket berperan untuk menghubungkan klien dan server agar data bisa dikirimkan dengan benar.

Dan TCP menjadi pilihan utama untuk komunikasi antara web server dan klien karena menawarkan keandalan dalam pengiriman data. Dengan TCP data yang dikirimkan akan sampai dengan tepat, urutannya benar, dan tidak ada yang hilang. Protokol ini juga mengatur aliran data agar tidak ada yang overload, yang menjadikannya pilihan yang sempurna untuk memastikan komunikasi yang stabil dan aman saat mengakses konten web.

2. Dalam konteks web server, apa yang dimaksud dengan kode status HTTP "404 Not Found"? Bagaimana cara web server menangani situasi ketika file yang diminta oleh klien tidak ditemukan? Jelaskan langkah-langkah yang harus dilakukan oleh server untuk memberikan respons yang sesuai!

Kode status HTTP "404 Not Found" berarti server tidak bisa menemukan halaman atau file yang diminta oleh klien, biasanya karena halaman tersebut sudah tidak ada atau alamat URL yang dimasukkan salah. Ketika hal ini terjadi, server akan terlebih dahulu memeriksa apakah file tersebut ada. Jika tidak server akan mengirimkan pesan "404 Not Found" kepada klien untuk memberi tahu pengguna bahwa halaman yang mereka cari tidak tersedia atau tidak ada.

3. Mengapa multithreading penting dalam pengembangan web server modern? Jelaskan bagaimana pendekatan multithreading dapat meningkatkan performa web server dalam menangani beberapa permintaan klien secara bersamaan! Apa tantangan yang mungkin muncul saat mengimplementasikan multithreading pada web server?

JARINGAN KOMPUTER – TUGAS PENDAHULUAN MODUL 8

WEB SERVER

Multithreading penting untuk web server modern karena memungkinkan server untuk menangani banyak permintaan klien sekaligus. Dengan memanfaatkan beberapa thread, setiap permintaan dapat diproses secara terpisah tanpa harus menunggu yang lain selesai. Ini meningkatkan kinerja server, membuatnya lebih responsif, dan mampu melayani banyak pengguna sekaligus.

Namun, mengimplementasikan multithreading juga membawa tantangan tersendiri, terutama dalam mengelola sumber daya seperti memori dan CPU. Jika tidak dikelola dengan baik, bisa muncul masalah seperti deadlock atau kebocoran memori. Koordinasi antar thread juga menjadi lebih rumit, sehingga perlu perhatian ekstra agar server tetap berjalan dengan lancar tanpa konflik antar thread.

4. Bagaimana web server menangani file statik seperti HTML, CSS, dan gambar? Jelaskan proses yang terjadi mulai dari permintaan klien hingga file dikirimkan ke klien. Apakah ada perbedaan dalam penanganan file statik dibandingkan dengan file dinamis?

Web server menangani file statik seperti HTML, CSS, dan gambar dengan mencari file tersebut di sistem file lokal setelah klien mengirimkan permintaan, lalu mengirimkannya langsung ke klien tanpa perlu pemrosesan tambahan. Sementara itu, untuk file dinamis, server perlu memprosesnya terlebih dahulu. Setelah permintaan diterima, server mengirimkannya ke aplikasi server-side (misalnya, PHP), yang akan mengolah data, seperti mengambil informasi dari database atau melakukan perhitungan, dan menghasilkan konten yang kemudian dikirimkan ke klien. Jadi, perbedaan utamanya adalah file statik langsung dikirim tanpa perubahan, sementara file dinamis memerlukan pemrosesan sebelum dikirim.

5. Jelaskan perbedaan utama antara web server dan web proxy. Apa fungsi utama dari masing-masing, dan bagaimana mereka bekerja sama dalam arsitektur jaringan?

Perbedaan utama antara web server dan web proxy terletak pada peran masing-masing dalam komunikasi jaringan. Web server berfungsi untuk menyimpan dan mengirimkan konten web, seperti halaman HTML atau gambar, langsung kepada klien setelah menerima permintaan. Server memproses permintaan tersebut dan mengirimkan respons sesuai dengan isi yang diminta oleh klien.

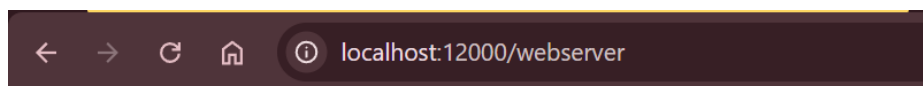
Sementara itu, web proxy berperan sebagai perantara antara klien dan server. Proxy menerima permintaan dari klien, meneruskannya ke server yang sesuai, lalu mengirimkan respons dari server kembali ke klien. Fungsi utama proxy adalah untuk meningkatkan keamanan, menyaring konten, atau menyembunyikan identitas klien. Keduanya bekerja bersama dalam jaringan, dengan proxy memberikan lapisan tambahan yang membantu mengoptimalkan dan mengamankan komunikasi antara klien dan server.

B. Web Server

1. Lengkapi skeleton code diatas! Lakukan eksekusi untuk menjalankan code yang sudah anda lengkapi dengan menggunakan browser! (hint: pada address bar ketik: http://localhost:{nomor_port}/{nama_file})

```
webserver.py
1  from socket import *
2  import sys
3
4  serverSocket = socket(AF_INET, SOCK_STREAM)
5
6  serverPort = 12000
7  serverSocket.bind(('', serverPort))
8  serverSocket.listen(1)
9
10 while True:
11     print('Ready to serve...')
12
13     connectionSocket, addr = serverSocket.accept()
14
15     try:
16         message = connectionSocket.recv(1024).decode()
17         filename = message.split()[1]
18
19         f = open(filename[1:])
20         outputdata = f.read()
21
22         connectionSocket.send('HTTP/1.1 200 OK\r\n'.encode())
23         connectionSocket.send('Content-Type: text/html\r\n\r\n'.encode())
24
25         for i in range(0, len(outputdata)):
26             connectionSocket.send(outputdata[i].encode())
27         connectionSocket.send("\r\n".encode())
28
```

```
28
29     connectionSocket.close()
30
31 except IOError:
32     connectionSocket.send('HTTP/1.1 404 Not Found\r\n'.encode())
33     connectionSocket.send('Content-Type: text/html\r\n\r\n'.encode())
34     connectionSocket.send('<html><body><h1>404 Not Found</h1></body></html>'.encode())
35
36     connectionSocket.close()
37
38 serverSocket.close()
39 sys.exit()
40
```



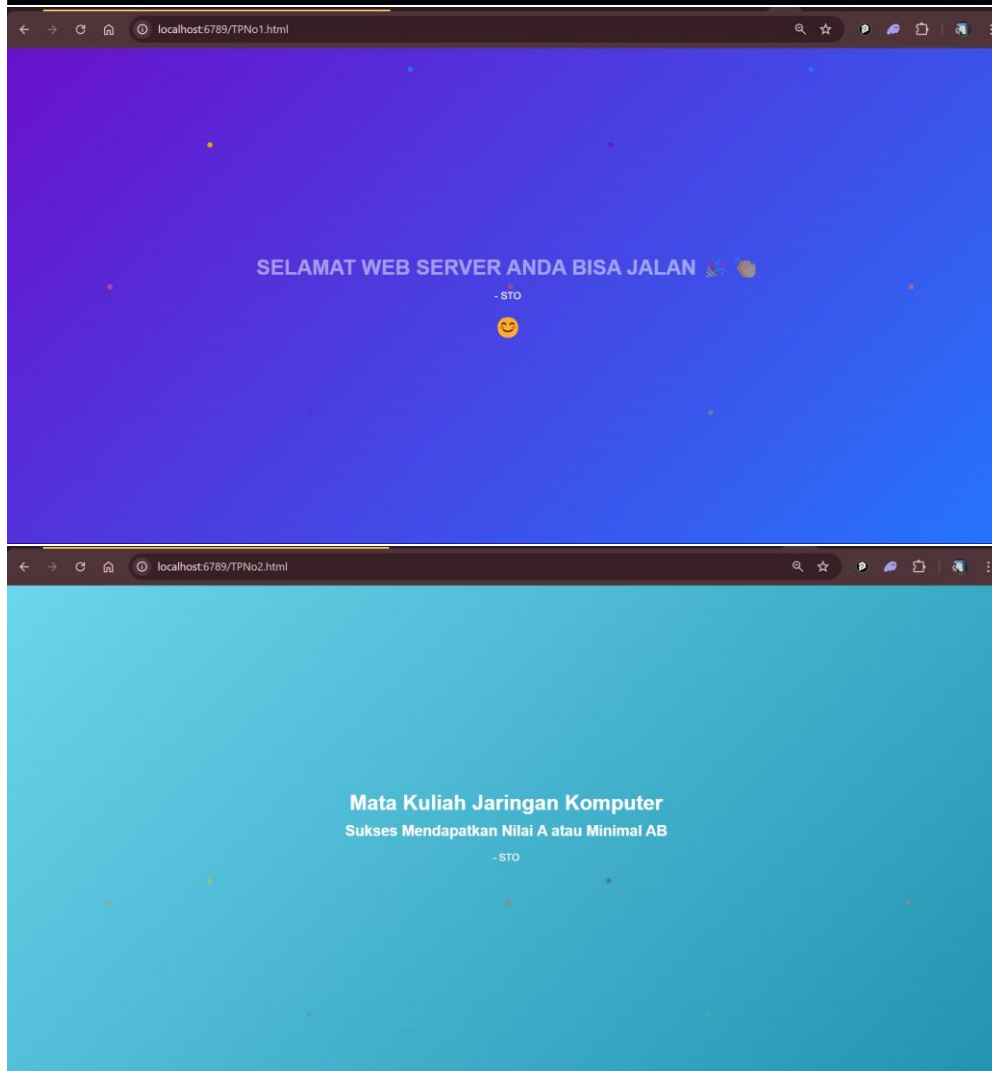
404 Not Found

2. Selanjutnya jika web server yang sudah anda lengkapi tersebut dapat di-run, maka selanjutnya akan dikembangkan lagi, karena server itu seharusnya tidak mungkin hanya melakukan satu kali request HTTP. Kembangkan web server tersebut agar dapat melayani beberapa request sekaligus. Manfaatkan threading! Berikut adalah kode threading yang mungkin akan membantu anda untuk mengembangkan web server sudah dikerjakan sebelumnya: Setelah anda menambahkan dan menyesuaikan threading pada kode tersebut, jalankan web server dan lakukan penerapan threading dengan mengakses 3 file yang sudah disediakan (TPNo1.html; TPNo2.html; dan TPNo3.html)

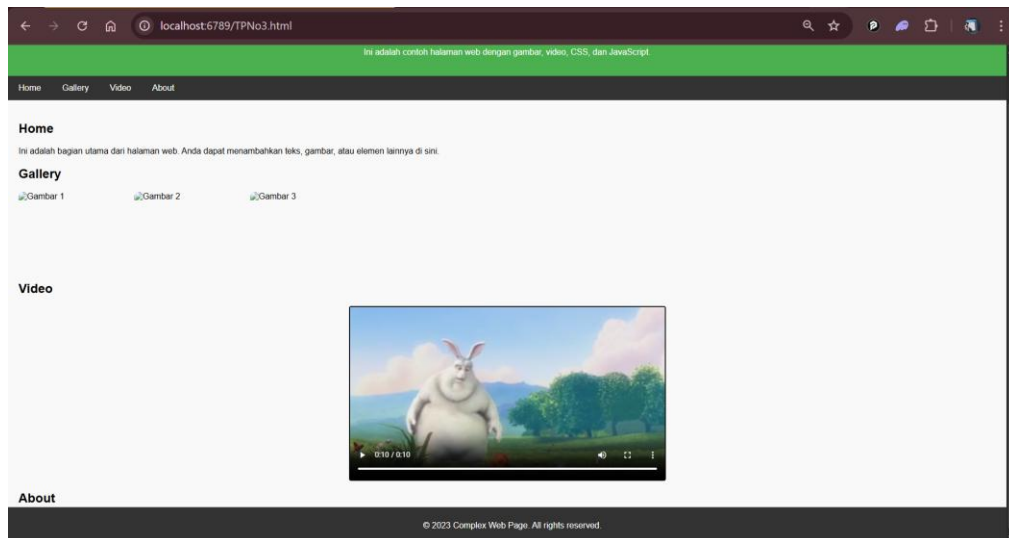
```
webserver.py x TPNo1.html
webserver.py
1  from socket import *
2  import sys
3  import threading
4
5  def handle_client(connectionSocket):
6      try:
7          message = connectionSocket.recv(1024).decode()
8
9          if len(message.split()) < 2:
10             connectionSocket.send("HTTP/1.1 400 Bad Request\r\n\r\n".encode())
11             connectionSocket.send("<html><body><h1>400 Bad Request</h1></body></html>\r\n".encode())
12             connectionSocket.close()
13             return
14
15             filename = message.split()[1]
16
17             try:
18                 with open(filename[1:], 'r', encoding='utf-8') as f:
19                     outputdata = f.read()
20
21                     connectionSocket.send("HTTP/1.1 200 OK\r\n\r\n".encode())
22
23                     for i in range(0, len(outputdata)):
24                         connectionSocket.send(outputdata[i].encode())
25                         connectionSocket.send("\r\n".encode())
26
27             except FileNotFoundError:
28
29
30             connectionSocket.close()
31
32             except Exception as e:
33                 print(f"Error: {e}")
34                 connectionSocket.send("HTTP/1.1 500 Internal Server Error\r\n\r\n".encode())
35                 connectionSocket.send("<html><body><h1>500 Internal Server Error</h1></body></html>\r\n".encode())
36                 connectionSocket.close()
37
38
39 serverSocket = socket(AF_INET, SOCK_STREAM)
40 serverSocket.bind(('', 6789))
41 serverSocket.listen(5)
42
43 print("Multithreaded server is ready to serve...")
44
45 while True:
46     connectionSocket, addr = serverSocket.accept()
47     print(f"Connection from {addr}")
48
49     client_thread = threading.Thread(target=handle_client, args=(connectionSocket,))
50     client_thread.start()
51
52 serverSocket.close()
53 sys.exit()
54
```

JARINGAN KOMPUTER – TUGAS PENDAHULUAN MODUL 8 WEB SERVER

```
PS C:\Exercise Program\WebServer> py webserver.py
Multithreaded server is ready to serve...
Connection from ('127.0.0.1', 53361)
Connection from ('127.0.0.1', 53362)
Connection from ('127.0.0.1', 53363)
Connection from ('127.0.0.1', 53365)
PS C:\Exercise Program\WebServer> py webserver.py
Multithreaded server is ready to serve...
Connection from ('127.0.0.1', 53361)
Connection from ('127.0.0.1', 53362)
PS C:\Exercise Program\WebServer> py webserver.py
Multithreaded server is ready to serve...
PS C:\Exercise Program\WebServer> py webserver.py
PS C:\Exercise Program\WebServer> py webserver.py
Multithreaded server is ready to serve...
PS C:\Exercise Program\WebServer> py webserver.py
Multithreaded server is ready to serve...
Connection from ('127.0.0.1', 53361)
Connection from ('127.0.0.1', 53362)
Connection from ('127.0.0.1', 53363)
Connection from ('127.0.0.1', 53363)
Connection from ('127.0.0.1', 53365)
Connection from ('127.0.0.1', 53367)
█
```



JARINGAN KOMPUTER – TUGAS PENDAHULUAN MODUL 8 WEB SERVER



3. Selanjutnya dari pada menguji server menggunakan browser, kenapa tidak menggunakan client saja! sehingga bagaimana cara membuat klien HTTP sederhana dalam Python yang menguji server dengan mengirim permintaan GET menggunakan koneksi TCP? Klien harus menerima argumen baris perintah berupa alamat IP atau nama host server, nomor port server, dan jalur file yang diminta. Setelah terhubung ke server, klien harus mengirim permintaan HTTP GET, menerima respons dari server, dan menampilkannya sebagai output. Format perintah untuk menjalankan klien adalah: `client.py server_host server_port filename` Misal jika file `index.html` ada di server dan server mengembalikan respons HTTP 200 OK dengan isi file tersebut: Untuk menguji client yang sudah anda buat jalankan web server dan cek file bernama `TPNo1.html`; `TPNo2.html`; `TPNo3.html` dan `TPNo4.html`!

```
● PS C:\Exercise Program\WebServer> python client.py localhost 6789 /index.html
HTTP/1.1 200 OK
```

```
● PS C:\Exercise Program\WebServer> python client.py localhost 6789 /TPNo1.html
HTTP/1.1 200 OK
```

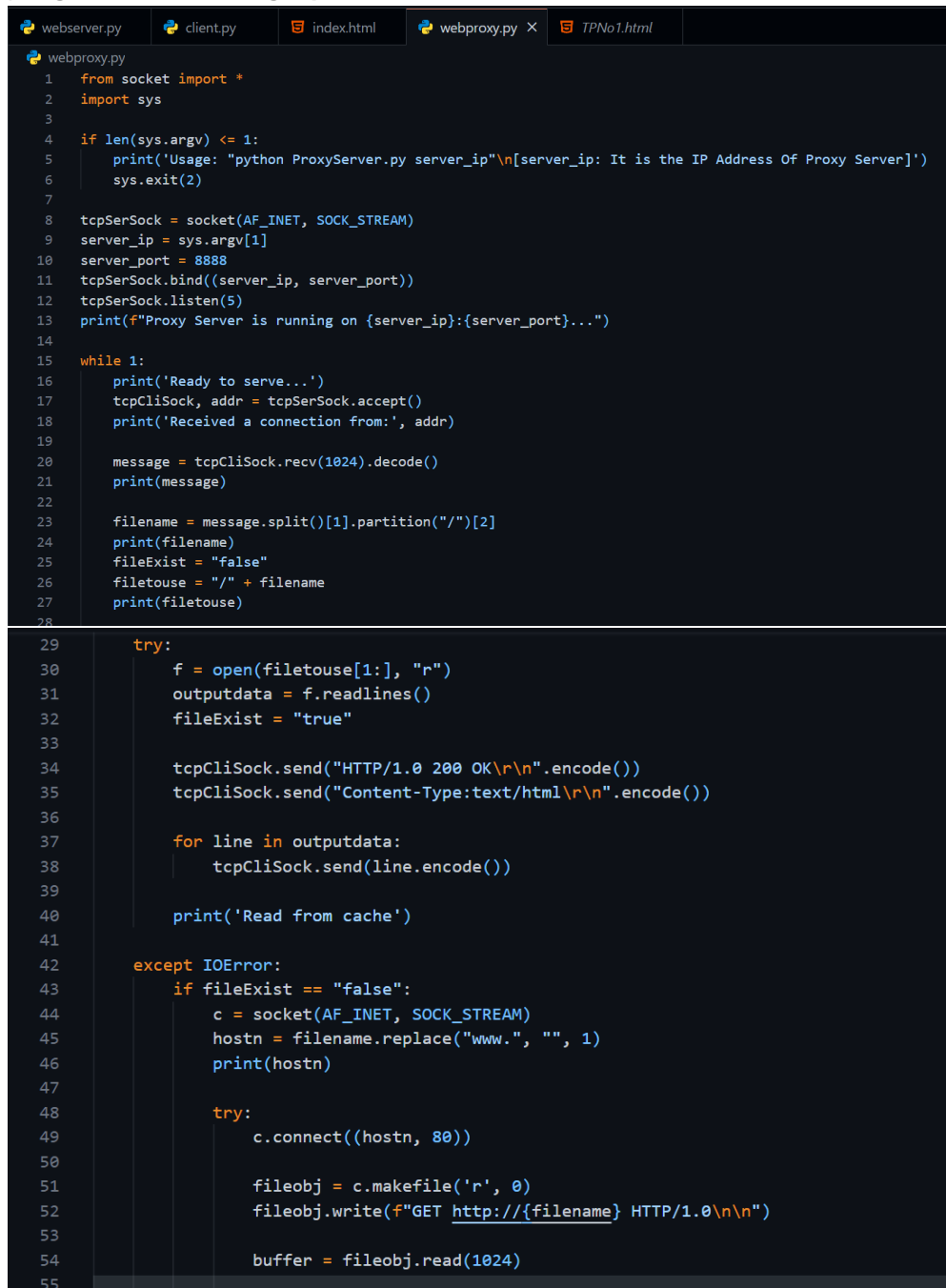
```
● PS C:\Exercise Program\WebServer> python client.py localhost 6789 /TPNo2.html
HTTP/1.1 200 OK
```

```
● PS C:\Exercise Program\WebServer> python client.py localhost 6789 /TPNo3.html
HTTP/1.1 200 OK
```

```
● PS C:\Exercise Program\WebServer> python client.py localhost 6789 /TPNo4.html
HTTP/1.1 404 Not Found
```

C. Web Proxy

1. Lengkapi skeleton code diatas! Lakukan eksekusi untuk menjalankan code yang sudah anda lengkapi!



```
webserver.py client.py index.html webproxy.py X TPN01.html
webproxy.py
1 from socket import *
2 import sys
3
4 if len(sys.argv) <= 1:
5     print('Usage: "python ProxyServer.py server_ip"\n[server_ip: It is the IP Address Of Proxy Server]')
6     sys.exit(2)
7
8 tcpSerSock = socket(AF_INET, SOCK_STREAM)
9 server_ip = sys.argv[1]
10 server_port = 8888
11 tcpSerSock.bind((server_ip, server_port))
12 tcpSerSock.listen(5)
13 print(f'Proxy Server is running on {server_ip}:{server_port}...')
14
15 while 1:
16     print('Ready to serve...')
17     tcpCliSock, addr = tcpSerSock.accept()
18     print('Received a connection from:', addr)
19
20     message = tcpCliSock.recv(1024).decode()
21     print(message)
22
23     filename = message.split()[1].partition("/")[2]
24     print(filename)
25     fileExist = "false"
26     fileouse = "/" + filename
27     print(fileouse)
28
29     try:
30         f = open(fileouse[1:], "r")
31         outputdata = f.readlines()
32         fileExist = "true"
33
34         tcpCliSock.send("HTTP/1.0 200 OK\r\n".encode())
35         tcpCliSock.send("Content-Type:text/html\r\n".encode())
36
37         for line in outputdata:
38             tcpCliSock.send(line.encode())
39
40         print('Read from cache')
41
42     except IOError:
43         if fileExist == "false":
44             c = socket(AF_INET, SOCK_STREAM)
45             hostn = filename.replace("www.", "", 1)
46             print(hostn)
47
48             try:
49                 c.connect((hostn, 80))
50
51                 fileobj = c.makefile('r', 0)
52                 fileobj.write(f"GET http://{filename} HTTP/1.0\r\n")
53
54                 buffer = fileobj.read(1024)
55
```

JARINGAN KOMPUTER – TUGAS PENDAHULUAN MODUL 8 WEB SERVER

```
54         buffer = fileobj.read(1024)
55
56         tmpFile = open(f"./{filename}", "wb")
57
58         while buffer:
59             tmpFile.write(buffer.encode())
60             tcpCliSock.send(buffer.encode())
61             buffer = fileobj.read(1024)
62
63         tmpFile.close()
64
65         except Exception as e:
66             print(f"Illegal request: {e}")
67             tcpCliSock.send("HTTP/1.0 404 Not Found\r\n".encode())
68             tcpCliSock.send("Content-Type:text/html\r\n".encode())
69             tcpCliSock.send("\r\n".encode())
70             tcpCliSock.send("<html><body><h1>404 Not Found</h1></body></html>\r\n".encode())
71
72         else:
73             tcpCliSock.send("HTTP/1.0 404 Not Found\r\n".encode())
74             tcpCliSock.send("Content-Type:text/html\r\n".encode())
75             tcpCliSock.send("\r\n".encode())
76             tcpCliSock.send("<html><body><h1>404 Not Found</h1></body></html>\r\n".encode())
77
78         tcpCliSock.close()
79
80     tcpSerSock.close()
81     sys.exit()
82
```

```
PS C:\Exercise Program\WebServer> python webproxy.py 127.0.0.1
Proxy Server is running on 127.0.0.1:8888...
Ready to serve...
Received a connection from: ('127.0.0.1', 56991)
Request: GET /TPNo1.html HTTP/1.0
Host: 127.0.0.1

Filename requested: TPNo1.html
File to use: /TPNo1.html
Connection was closed by the remote host while sending data.
Read from cache
Ready to serve...
[]

PS C:\Exercise Program\WebServer> python client.py 127.0.0.1 8888 /TPNo1.html
HTTP/1.0 200 OK
```

2. Apa yang membedakan kode diatas (Web Proxy) dengan kode sebelumnya (WebServer)? Apakah mungkin sebuah web server menjadi web proxy juga?

Web server berfungsi untuk menyimpan dan mengelola konten web, serta memberikan respons atas permintaan klien dengan mengirimkan file yang diminta. Web server hanya melayani file yang ada di sistem lokal dan tidak bertindak sebagai perantara untuk meneruskan permintaan dari klien ke server lain. Sedangkan web proxy bertindak sebagai perantara antara klien dan web server. Proxy menerima permintaan dari klien, meneruskannya ke server yang tepat, dan mengirimkan respons kembali ke klien. Proxy juga dapat menyimpan cache untuk mempercepat akses ke file yang sering diminta, serta digunakan untuk tujuan keamanan dan kontrol akses.

JARINGAN KOMPUTER – TUGAS PENDAHULUAN MODUL 8 WEB SERVER

Meskipun memiliki fungsi yang berbeda, sebuah web server sebenarnya bisa juga berfungsi sebagai web proxy jika dilengkapi dengan fitur tambahan seperti reverse proxy atau forward proxy. Dengan fitur tersebut, web server bisa meneruskan permintaan klien ke server lain, misalnya untuk load balancing atau pengelolaan konten dinamis. Beberapa web server, seperti Nginx dan Apache, sudah memiliki kemampuan ini, memungkinkan mereka untuk mengarahkan permintaan ke server lain sambil tetap melayani file statik.

3. Pada kode diatas belum ada error handling, sehingga jika diberikan kode error handling berikut: akan disimpan dimana dan dibaris nomor berapa?

Error handling untuk "404 Not Found" harus ditambahkan setelah baris 55 (setelah `buffer = fileobj.read(1024)`) dan sebelum melanjutkan pengiriman data ke klien. Ini memastikan bahwa jika respons dari server web asal adalah "404 Not Found", proxy langsung mengirimkan 404 ke klien dan menghentikan pengolahan lebih lanjut.

4. Berikan pendapat anda apa yang terjadi pada web server jika ditambahkan post method?

Menambahkan metode POST pada web server memungkinkan server untuk menerima dan memproses data dari klien, seperti informasi dari formulir, login, atau file yang diunggah. Berbeda dengan GET, POST mengirimkan data melalui badan permintaan, yang membuatnya lebih fleksibel dan aman karena tidak menampilkan data dalam URL. Dengan POST, server bisa memvalidasi data, menyimpannya ke database, atau mengolah informasi lebih lanjut. Ini berguna untuk aplikasi yang memerlukan pengiriman data sensitif, dan tentu saja, menggunakan HTTPS penting untuk menjaga keamanan data selama proses pengiriman.

D. Analisa Code

- 1. Bagaimana cara server menangani masalah encoding dalam konteks membaca file dan mengirim respons ke klien? Apa yang akan terjadi jika file yang diminta oleh klien berisi karakter non-ASCII (misalnya, karakter Unicode seperti ä, ß, atau 日本語)? Jelaskan langkah-langkah yang dilakukan oleh server untuk memastikan kompatibilitas dengan karakter tersebut.**

Server menangani masalah encoding dengan membuka file yang diminta oleh klien menggunakan encoding UTF-8 pada saat membaca file (baris 53). Dengan menggunakan encoding ini, server dapat mengelola karakter-karakter non-ASCII atau Unicode, seperti huruf aksent (ä, ß) atau bahkan karakter dari bahasa lain seperti Jepang (日本語). Setelah file dibaca, server mengkodekan isi file menjadi byte dengan format UTF-8 sebelum mengirimkan data tersebut ke klien. Selain itu, header respons HTTP juga disertakan dengan Content-Type: text/html; charset=utf-8, yang memberi tahu klien (misalnya, browser) untuk memproses dan menampilkan karakter-karakter tersebut dengan benar. Dengan langkah-langkah ini, server memastikan bahwa karakter non-ASCII ditangani dengan tepat, dan klien dapat melihatnya tanpa masalah.

- 2. Mengapa server menggunakan metode sendall() alih-alih send() untuk mengirim respons ke klien? Apa yang akan terjadi jika send() digunakan dan data yang dikirim lebih besar dari buffer klien**

Server menggunakan metode sendall() untuk memastikan bahwa seluruh data yang dikirimkan sampai ke klien tanpa terputus. Metode sendall() secara otomatis menangani pengiriman data dalam potongan-potongan kecil jika data yang dikirim lebih besar dari buffer klien, sehingga menjamin bahwa semua data akan terkirim dengan benar. Jika send() digunakan, dan data yang dikirim lebih besar dari buffer, maka send() hanya mengirimkan sebagian data dan mengembalikan jumlah byte yang berhasil dikirim. Hal ini bisa menyebabkan sisa data tidak terkirim, dan perlu penanganan tambahan untuk mengirimkan sisa data tersebut. Oleh karena itu, sendall() lebih aman dan mudah digunakan ketika mengirimkan data dalam jumlah besar, karena ia memastikan bahwa data dikirimkan secara lengkap tanpa terlewat.